

COMSATS University Islamabad, Lahore Campus

Course Title:	Lab-DevOps for Cloud Computing			Course Code:		Credit Hours:	01
Course Instructor:	M Khizar Hayat			Program Name:	BSCS		
Semester:	6	Batch:		Section:	C	Date:	
Time Allowed:				Maximum Marks:	50		
Student's Name:				Reg. No.			

Important Instructions / Guidelines:

1. This is individual Final Project
2. Each team must follow Git Flow and GitHub collaborative best practices.
3. Must contribute by developing at least one complete HTML and CSS web page and at least one CI or CD workflow
4. Three GitHub Environments must be created: development, staging, and production.
5. Separate CI and CD pipelines must exist for each environment.
6. Docker must be used to containerize the static website.
7. Secrets must be managed using GitHub Environments and Render environment variables.
8. Only the Team Lead will submit this completed document.
9. Github Repo must be "Public"

Details

Sr #	Name	Roll Number	Role	Assigned Web Page	Workflow Created
1			Team Lead	Home Page	

Objectives

This project involves designing and implementing a cloud-native microservices application for a fictional e-commerce platform. The application consists of multiple services (User, Product, Order, and Notification Services) and a frontend dashboard. Each service is containerized using Docker, orchestrated with Kubernetes, and deployed via a CI/CD pipeline using Jenkins, simulating an enterprise-level cloud DevOps workflow.

Group Repository URL	
Production Environment Website URL	
QA Environment Website URL	
Development Environment Website URL	

Scenario:

You are part of a DevOps team tasked with delivering a scalable, automated deployment solution for the company's e-commerce platform. Developers push updates frequently, and the platform must remain highly available and fault-tolerant. The DevOps pipeline should ensure:

Automated builds and tests for each microservice.

Containerization and versioning for consistent deployment.

Deployment and orchestration on a Kubernetes cluster with zero downtime.

Continuous monitoring and logging of services.

Security and environment isolation using ConfigMaps, Secrets, and RBAC

Folder Structure (Example)

cloud-Cloud-Native Microservices DevOps Project-project/

- .github/workflows/ci.yml # GitHub Actions CI/CD workflows
- .gitignore # Ignore unnecessary files
- .dockerignore # Ignore files for Docker build
- Dockerfile # Dockerfile for frontend or microservice
- Jenkinsfile # Jenkins declarative pipeline
- README.md # Project documentation
- src/
 - frontend/ # Frontend dashboard
 - user-service/ # User service source code
 - product-service/ # Product service source code
 - order-service/ # Order service source code
 - notification-service/ # Notification service source code
- k8s/
 - user-deployment.yaml # Kubernetes Deployment for User Service
 - product-deployment.yaml # Kubernetes Deployment for Product Service
 - order-deployment.yaml # Kubernetes Deployment for Order Service
 - notification-deployment.yaml # Kubernetes Deployment for Notification Service
 - frontend-deployment.yaml # Kubernetes Deployment for Frontend

Jenkins Pipeline (Example)

Stages: Checkout → Build → Test → Docker Build → Push → Deploy → Notify

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build & Test') {
      steps { sh 'npm install && npm test' }
    }
    stage('Docker Build & Push') {
      steps {
        sh 'docker build -t myrepo/user-service:latest ./user-service'
        sh 'docker push myrepo/user-service:latest'
      }
    }
    stage('Deploy to Kubernetes') {
      steps { sh 'kubectl apply -f k8s/user-deployment.yaml' }
    }
    stage('Notification') {
      steps { echo 'Deployment Successful!' }
    }
  }
}
```

Required Web Pages and Content

- **User Service:** Handles user registration, login, profiles, and authentication. Could use JWT for authentication; REST APIs exposed for frontend.
- **Product Service:** Manages product catalog, details, and inventory. Provides CRUD APIs; can include search and filtering functionality.
- **Order Service:** Processes orders, tracks order status, and communicates with Payment & Product Services. Must handle multiple API calls; transactional behavior recommended.
- **Notification Service:** Sends email or in-app notifications for events like order confirmation or alerts. Could integrate with SMTP/email API or message queue like RabbitMQ.

Key Features & Topics Covered

Containerization

Create Dockerfiles for each microservice.
Build and tag images with semantic versioning.
Push images to a Docker registry for centralized management.

Kubernetes

Deploy services using Deployments and StatefulSets.
Configure Services, Ingress, and Network Policies.
Manage environment-specific variables with ConfigMaps and Secrets.
Implement rolling updates and autoscaling.

CI/CD Pipeline

Use Jenkins declarative pipeline to automate build, test, and deploy.
Integrate unit tests and linting for code quality.
Trigger pipeline on Git commits and pull requests.
Automate Docker image build and deployment to Kubernetes.

Jenkins Pipeline

Multi-stage pipeline: Checkout → Build → Test → Docker Build → Push → Deploy → Notification.
Implement **Red-Green-Refactor** style automated testing.
Provide failure notifications via Slack or email.
Include rollback steps on deployment failure.

Advanced Practices

Blue-Green or Canary deployments in Kubernetes.
Centralized logging and monitoring (e.g., using Prometheus and Grafana).
Branch protection and Git workflow for collaboration.

Core DevOps Requirements

- GitHub repository using Git Flow (develop, release, production branches)
- Dockerfile to containerize the static website
- CI pipelines for HTML/CSS linting and Docker image build
- CD pipelines deploying to separate Render.com services per environment
- GitHub Environments with protected secrets
- Automated deployments triggered by branch activity

Evaluation Criteria (50 Marks)

Repository & Structure Organized Git repository, folder structure, Kubernetes manifests. **5 Marks**
Containerization (Docker) Dockerfiles for all microservices, image builds, tagging, local testing. **7 Marks**
CI/CD Pipeline (Jenkins) Multi-stage Jenkins pipeline automating build, test, Docker build & deploy. **8 Marks**
Kubernetes Deployment Deployments, Services, ConfigMaps, Secrets, rolling updates. **8 Marks**
Code Quality & Testing Unit tests, linting, Red-Green-Refactor approach. **5 Marks**
Branching & Collaboration Feature branches, PR workflow, branch protection. **3 Marks**
Documentation & README Setup instructions, pipeline explanation, repository overview. **3 Marks**
Local Verification Screenshots/proof of builds, tests, Docker runs, Kubernetes deployment. **3 Marks**

Deployment Success Fully functional deployment; services accessible and communicating.**5 Marks**
Innovation / Advanced Features(Optional) Blue-Green/Canary deployment, rollback, autoscaling, secure configs.**3 Marks**

Submission Requirements

- GitHub repository URL.
- Docker Hub repository URL.
- Screenshots for local verification from all team members.
- Short reflection (3–4 lines) describing learnings and workflow challenges.

Learning Outcomes

- Hands-on experience with containerized microservices and Kubernetes orchestration.
- Implementing a complete CI/CD pipeline with Jenkins.
- Understanding enterprise-grade DevOps practices: testing, monitoring, rollback, and secure configuration.
- Deploying cloud-native applications with scalability, fault tolerance, and zero downtime.